./install.sh must be run twice. It stops during step 1 the first time.

Why It Stopped During Step 1 on the First Run

The failure was caused by the combination of bash's strict error handling (`set -euo pipefail` at the top of `install.sh`) and the subshell command used to update crontab:

```
(crontab -l 2>/dev/null; echo "${CRON_JOB}") | crontab -
```

1. **On a fresh server, root has no existing crontab file**: When no crontab exists yet, `crontab -l` exits with exit code **`1`** (`"no crontab for root"`).
2. Because `set -e` was active, when `crontab -l` returned `1`, bash immediately terminated the subshell before it could execute `echo "${CRON_JOB}"`.
3. Because `set -o pipefail` was active, the failed subshell marked the entire pipeline as failed with code `1`, causing `install.sh` to abort right there in Step 1.
4. **Why it succeeded on the second run**: On the second run (or after `crontab -` had initialized the spool file), `crontab -l` found an existing file and returned exit code `0`, allowing the subshell to complete without triggering `set -e`.

The Fix

Step 1 in install.sh has been rewritten to be fail-safe on fresh servers:

1. **Checks/installs `cron` if missing**: Verifies `crontab` exists before running (and installs the `cron` package if on a minimal image).
2. **Safely captures existing crontab**: Uses `CURRENT_CRON="$(crontab -l 2>/dev/null || true)"` so that an empty/missing crontab evaluates cleanly to an empty string with exit code `0`.
3. **Appends cleanly**: Pipes `CURRENT_CRON` + `CRON_JOB` directly without fragile subshells:

```
printf "%s\n%s\n" "${CURRENT_CRON}" "${CRON_JOB}" | sed '/^[[:space:]]*$/d' |
crontab -
```

The script now completes smoothly on the **first run** without stopping. This fix has been committed to the repository's `main` branch.